LAB IMPLEMENTATION 10
Hyperledger Fabric Network and Asset Management

BC

Aim

To configure a Hyperledger Fabric test network, deploy JavaScript chaincode, and perform asset initialization, querying, creation, and ownership transfer.

Theory

Hyperledger Fabric is a permissioned blockchain platform in which known organizations operate peers and endorse transactions according to a channel's policies. A channel provides a private logical ledger shared by its participating members. The ledger records ordered transactions, while the current state database provides the latest values for assets. Chaincode is the application logic that reads and updates ledger state through a Fabric transaction context. In this experiment, a JavaScript smart contract represents assets as JSON objects containing an identifier, color, size, owner, and appraised value. Initialization writes sample assets to the ledger. Query functions retrieve one asset or a collection of assets, while creation adds a new record and transfer changes an asset's owner. Fabric orders and validates submitted transactions before committed updates become part of the ledger state. The chaincode used in this experiment is included in the report as source code and was deployed and executed on the network as shown in the output.

Steps

1. Check the installed tools and verify the required packages, including jq, curl, git, Node.js and npm.
2. Clone the hyperledger/fabric-samples repository and download the Hyperledger Fabric v2.5.10 binaries (peer, orderer, configtxgen and cryptogen).
3. Set PATH and FABRIC_CFG_PATH to the Fabric binaries and configuration directories, and verify the CLI using peer version.
4. Generate the organizations' cryptographic material with cryptogen, then create the genesis block and the channel transaction artifacts with configtxgen.
5. Start the orderer and the two organization peers as local processes (the peers run in chaincode development mode). The network for this run was restarted on the existing persisted ledger, preserving the previously committed state.
6. Create the channel mychannel, join both peers to it, and submit the anchor peer updates for Org1 and Org2.
7. Install the JavaScript asset-transfer chaincode named basic on both peers and instantiate it on mychannel; start the chaincode processes so they register with their peers.
8. Initialize the ledger by invoking InitLedger (already committed on the persisted ledger for this run).
9. Query all assets with GetAllAssets (nine assets present), and confirm the new asset id is unused.
10. Invoke CreateAsset to add a new asset (asset10) with owner Rutuja Bhalekar and TransferAsset to change its owner to Lucas; query the record on both peers to inspect its updated state.

Code

```
/*
 * Copyright IBM Corp. All Rights Reserved.
 *
 * SPDX-License-Identifier: Apache-2.0
 */

```

```
'use strict';

// Deterministic JSON.stringify()
const stringify  = require('json-stringify-deterministic');
const sortKeysRecursive  = require('sort-keys-recursive');
const { Contract } = require('fabric-contract-api');

class AssetTransfer extends Contract {

    async InitLedger(ctx) {
        const assets = [
            {
                ID: 'asset1',
                Color: 'blue',
                Size: 5,
                Owner: 'Tomoko',
                AppraisedValue: 300,
            },
            {
                ID: 'asset2',
                Color: 'red',
                Size: 5,
                Owner: 'Brad',
                AppraisedValue: 400,
            },
            {
                ID: 'asset3',
                Color: 'green',
                Size: 10,
                Owner: 'Jin Soo',
                AppraisedValue: 500,
            },
            {
                ID: 'asset4',
                Color: 'yellow',
                Size: 10,
                Owner: 'Max',
                AppraisedValue: 600,
            },
            {
                ID: 'asset5',
                Color: 'black',
                Size: 15,
                Owner: 'Adriana',
                AppraisedValue: 700,
            },
            {
                ID: 'asset6',
                Color: 'white',
                Size: 15,
                Owner: 'Michel',
                AppraisedValue: 800,
            },
        ];

        for (const asset of assets) {
            asset.docType = 'asset';
            // example of how to write to world state deterministically
            // use convetion of alphabetic order
            // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
            // when retrieving data, in any lang, the order of data will be the same and consequently also the corresonding hash
            await ctx.stub.putState(asset.ID, Buffer.from(stringify(sortKeysRecursive(asset))));
        }
    }

    // CreateAsset issues a new asset to the world state with given details.
    async CreateAsset(ctx, id, color, size, owner, appraisedValue) {
        const exists = await this.AssetExists(ctx, id);
        if (exists) {
            throw new Error(`The asset ${id} already exists`);
        }

        const asset = {
            ID: id,
            Color: color,
            Size: Number(size),
            Owner: owner,
            AppraisedValue: Number(appraisedValue),
        };
        // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
        await ctx.stub.putState(id, Buffer.from(stringify(sortKeysRecursive(asset))));
        return JSON.stringify(asset);
    }

    // ReadAsset returns the asset stored in the world state with given id.
    async ReadAsset(ctx, id) {
        const assetJSON = await ctx.stub.getState(id); // get the asset from chaincode state
        if (!assetJSON || assetJSON.length === 0) {
            throw new Error(`The asset ${id} does not exist`);
        }
```

```
        return assetJSON.toString();
    }

    // UpdateAsset updates an existing asset in the world state with provided parameters.
    async UpdateAsset(ctx, id, color, size, owner, appraisedValue) {
        const exists = await this.AssetExists(ctx, id);
        if (!exists) {
            throw new Error(`The asset ${id} does not exist`);
        }

        // overwriting original asset with new asset
        const updatedAsset = {
            ID: id,
            Color: color,
            Size: size,
            Owner: owner,
            AppraisedValue: appraisedValue,
        };
        // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
        return ctx.stub.putState(id, Buffer.from(stringify(sortKeysRecursive(updatedAsset))));
    }

    // DeleteAsset deletes an given asset from the world state.
    async DeleteAsset(ctx, id) {
        const exists = await this.AssetExists(ctx, id);
        if (!exists) {
            throw new Error(`The asset ${id} does not exist`);
        }
        return ctx.stub.deleteState(id);
    }

    // AssetExists returns true when asset with given ID exists in world state.
    async AssetExists(ctx, id) {
        const assetJSON = await ctx.stub.getState(id);
        return assetJSON && assetJSON.length > 0;
    }

    // TransferAsset updates the owner field of asset with given id in the world state.
    async TransferAsset(ctx, id, newOwner) {
        const assetString = await this.ReadAsset(ctx, id);
        const asset = JSON.parse(assetString);
        const oldOwner = asset.Owner;
        asset.Owner = newOwner;
        // we insert data in alphabetic order using 'json-stringify-deterministic' and 'sort-keys-recursive'
        await ctx.stub.putState(id, Buffer.from(stringify(sortKeysRecursive(asset))));
        return oldOwner;
    }

    // GetAllAssets returns all assets found in the world state.
    async GetAllAssets(ctx) {
        const allResults = [];
        // range query with empty string for startKey and endKey does an open-ended query of all assets in the chaincode namespace.
        const iterator = await ctx.stub.getStateByRange('', '');
        let result = await iterator.next();
        while (!result.done) {
            const strValue = Buffer.from(result.value.value.toString()).toString('utf8');
            let record;
            try {
                record = JSON.parse(strValue);
            } catch (err) {
                console.log(err);
                record = strValue;
            }
            allResults.push(record);
            result = await iterator.next();
        }
        return JSON.stringify(allResults);
    }
}

module.exports = AssetTransfer;
```

Listing 1. assetTransfer.js — asset-transfer-basic JavaScript chaincode

Output

```
afnan@ubuntu:~$ peer channel getinfo -c mychannel
2026-10-29 09:37:40.728 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":12,"currentBlockHash":"dUs52EKFCCHtuL01T+hkVgXCA+uEIIhTZOsMjUZ1BoY=","previousBlockHash":"G
qzqrWYBi9twZsjpv1/BmFwUnqHi9E8zpjcwzjYWyKY="}
afnan@ubuntu:~$
```

Fig. 1. Channel information for mychannel before this run (block height 12)

```
afnan@ubuntu:~$ peer chaincode list --installed 2>&1 | tail -n 1; peer chaincode list --instantiated -C mychannel 2>&1
 | tail -n 1
Name: basic, Version: 1.0, Path: /home/hatch/workspace/fabric_exp10/chaincode-src, Id: 41e23b8e139720573b56c036c88afc4
83c1c27829a8d372c37ab845a6da8887b
Name: basic, Version: 1.0, Path: /home/hatch/workspace/fabric_exp10/chaincode-src, Escc: escc, Vscc: vscc
afnan@ubuntu:~$
```

Fig. 2. JavaScript asset-transfer chaincode installed and instantiated on mychannel

```
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"GetAllAssets\"]}"
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400
,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"as
set3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max
","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docTyp
e":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"},{"Apprai
sedValue":900,"Color":"blue","ID":"asset7","Owner":"Lucas","Size":20},{"AppraisedValue":900,"Color":"blue","ID":"asset
8","Owner":"Lucas","Size":20},{"AppraisedValue":900,"Color":"blue","ID":"asset9","Owner":"Lucas","Size":20}]
afnan@ubuntu:~$
```

Fig. 3. GetAllAssets query listing the nine assets already on the ledger

```
afnan@ubuntu:~$ peer chaincode invoke -o orderer.example.com:7050 -C mychannel -n basic --peerAddresses peer0.org1.exa
mple.com:7051 --peerAddresses peer0.org2.example.com:9051 -c "{\"function\":\"CreateAsset\",\"Args\":[\"asset10\",\"bl
ue\",\"20\",\"Rutuja Bhalekar\",\"900\"]}" 2>&1 | tail -n 2
2026-10-29 09:37:42.975 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: st
atus:200 payload:"{\"ID\":\"asset10\",\"Color\":\"blue\",\"Size\":20,\"Owner\":\"Rutuja Bhalekar\",\"AppraisedValue\":
900}"
afnan@ubuntu:~$
```

Fig. 4. CreateAsset transaction creating asset10 with owner Rutuja Bhalekar

```
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"ReadAsset\",\"asset10\"]}"
{"AppraisedValue":900,"Color":"blue","ID":"asset10","Owner":"Rutuja Bhalekar","Size":20}
afnan@ubuntu:~$
```

Fig. 5. ReadAsset query showing the newly created asset10 (owner Rutuja Bhalekar)

```
afnan@ubuntu:~$ peer chaincode invoke -o orderer.example.com:7050 -C mychannel -n basic --peerAddresses peer0.org1.exa
mple.com:7051 --peerAddresses peer0.org2.example.com:9051 -c "{\"function\":\"TransferAsset\",\"Args\":[\"asset10\",\"
Lucas\"]}" 2>&1 | tail -n 2
2026-10-29 09:37:48.481 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: st
atus:200 payload:"Rutuja Bhalekar"
afnan@ubuntu:~$
```

Fig. 6. TransferAsset transaction transferring asset10 to Lucas

```
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"GetAllAssets\"]}"; peer channel getinfo -c
 mychannel
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":900
,"Color":"blue","ID":"asset10","Owner":"Lucas","Size":20},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"B
rad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docT
ype":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"Apprai
sedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Colo
r":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"},{"AppraisedValue":900,"Color":"blue","ID":"asse
t7","Owner":"Lucas","Size":20},{"AppraisedValue":900,"Color":"blue","ID":"asset8","Owner":"Lucas","Size":20},{"Apprais
edValue":900,"Color":"blue","ID":"asset9","Owner":"Lucas","Size":20}]
2026-10-29 09:37:51.704 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":14,"currentBlockHash":"jrZV4olO6XDxuXq6R+8/VZ29NmeUbMrucxmSivNzzXs=","previousBlockHash":"M
NPImXQiCePvu+8XtuYiAHwUW0N+TYq1L36gJtSOLfY="}
afnan@ubuntu:~$
```

Fig. 7. GetAllAssets after the transfer (ten assets) and channel height 14

```
afnan@ubuntu:~$ peer chaincode query -C mychannel -n basic -c "{\"Args\":[\"ReadAsset\",\"asset10\"]}"; setOrg2; peer
chaincode query -C mychannel -n basic -c "{\"Args\":[\"ReadAsset\",\"asset10\"]}"
{"AppraisedValue":900,"Color":"blue","ID":"asset10","Owner":"Lucas","Size":20}
{"AppraisedValue":900,"Color":"blue","ID":"asset10","Owner":"Lucas","Size":20}
afnan@ubuntu:~$
```

Fig. 8. ReadAsset verification on the Org1 and Org2 peers showing owner Lucas

Conclusion

The Hyperledger Fabric test network was configured and the JavaScript asset-transfer chaincode was deployed on the channel mychannel and executed end to end. The GetAllAssets query returned the committed ledger state, a new asset asset10 was created with owner Rutuja Bhalekar through the CreateAsset transaction, and the TransferAsset transaction changed its owner to Lucas, which was verified by querying the record on both the Org1 and Org2 peers. The channel height advanced from 12 to 14 during this run. The experiment thus demonstrates ledger initialization, asset creation, querying and ownership transfer on a permissioned Fabric network.